

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования

ОТЧЁТ

по практическому занятию № 3

по дисциплине «Создание программного обеспечения»

Тема: «Проектирование базы данных монолитного веб-приложения:
концептуальная и логическая модель»

Выполнил:

студент группы ЭФБО-17-24

Васин С. А. _____

Проверил:

Макиевский С. Е. _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Цель работы	3
Задание	3
1 ХОД РАБОТЫ	4
1.1 Основы проектирования баз данных	4
1.2 Требования и анализ данных	4
1.3 Проектирование схемы базы данных	5
1.3.1 Таблицы, атрибуты и ключи	7
1.3.2 Связи и целостность	9
1.3.3 Выбор типов	9
1.4 Оптимизация производительности	9
1.5 Нормализация и денормализация	10
1.6 Безопасность данных	11
2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ	13
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15

ВВЕДЕНИЕ

Цель работы

Спроектировать базу данных GPU Rent — сервиса аренды облачных GPU-серверов. По требованиям П2 определить сущности и связи, построить концептуальную и логическую модели, выбрать типы и ограничения, обосновать нормализацию, оптимизацию и защиту данных.

Задание

1. Изучить сущности, атрибуты, отношения и нормализацию.
2. Определить необходимые данные по требованиям приложения.
3. Создать схему таблиц, связей, ключей, типов, индексов и ограничений.
4. Рассмотреть индексирование, кэширование и агрегирование.
5. Обосновать нормализацию и денормализацию.
6. Предусмотреть ограничения доступа и шифрование.

П3 описывает модель и решения проектирования; П4 — физическую реализацию, тесты и измерения. Фактическая схема реализована миграциями db/migrations/0001-0012, серверная и клиентская части ещё запланированы.

1 ХОД РАБОТЫ

1.1 Основы проектирования баз данных

Сущность — объект предметной области, атрибут — его свойство, связь — отношение между объектами. Концептуальная модель показывает смысл данных; логическая определяет таблицы, ключи и зависимости; физическая описывает реализацию в конкретной СУБД [1, 2].

Первичный ключ (РК) однозначно определяет строку, внешний (FK) ссылается на существующую строку другой таблицы. Связь 1:N задаётся FK на стороне «много», 1:1 требует также UNIQUE, M:N реализуется промежуточной таблицей. Обязательные значения защищает NOT NULL, диапазоны и сочетания — CHECK.

Нормализация устраняет аномалии вставки, изменения и удаления: 1НФ требует атомарных значений, 2НФ исключает зависимость от части составного ключа, 3НФ — транзитивные зависимости неключевых атрибутов [2, 3]. Денормализация допустима ради конкретного запроса, если определены источник истины и механизм согласования.

1.2 Требования и анализ данных

Клиент регистрируется, добавляет ключи доступа, пополняет баланс, выбирает площадку и GPU, запускает инстанс из шаблона. Администратор управляет оборудованием и ценами; учёт потребления формирует списания и аналитику. Требования П2 переведены в группы данных:

Требования П2	Что необходимо хранить
FR-01–FR-04: учётная запись и доступ	Email, хэш пароля, роль, статус, публичные SSH-ключи, хэши API-ключей
FR-05, FR-14, FR-15: каталог и управление	Дата-центры, ноды, физические GPU, модели, цены и периоды действия
FR-06–FR-09: запуск и хранение	Шаблон, параметры инстанса, выделенные GPU, том, секреты окружения
FR-10–FR-13: расчёты	Интервалы потребления, зафиксированная цена, платежи, проводки, баланс
FR-17, FR-18: аналитика и аудит	Выручка по дням, автор действия, объект, время и изменённые значения

Отзыв API-ключа даёт `revoked_at`, повторный `webhook` — уникальный идентификатор платежа, посекундная оплата — границы интервала и снимок цены.

Цена зависит от площадки, тарифа и времени. Платёж, потребление и проводка разделяют подтверждение оплаты, использование ресурса и изменение баланса. Том существует независимо от инстанса; интервалы занятости GPU сохраняют историю выделения.

1.3 Проектирование схемы базы данных

Концептуальная модель объединяет пользователей и деньги, аренду и инфраструктуру. Нотация «воронья лапка» показывает кардинальность и обязательность связей.

Логическая модель показана двумя диаграммами: доступ и расчёты; оборудование, цены и аренда.

Логическая ER-модель, часть 1: пользователи, платежи и биллинг
 (PK — первичный ключ, FK — внешний, UK — уникальный, * — NOT NULL)

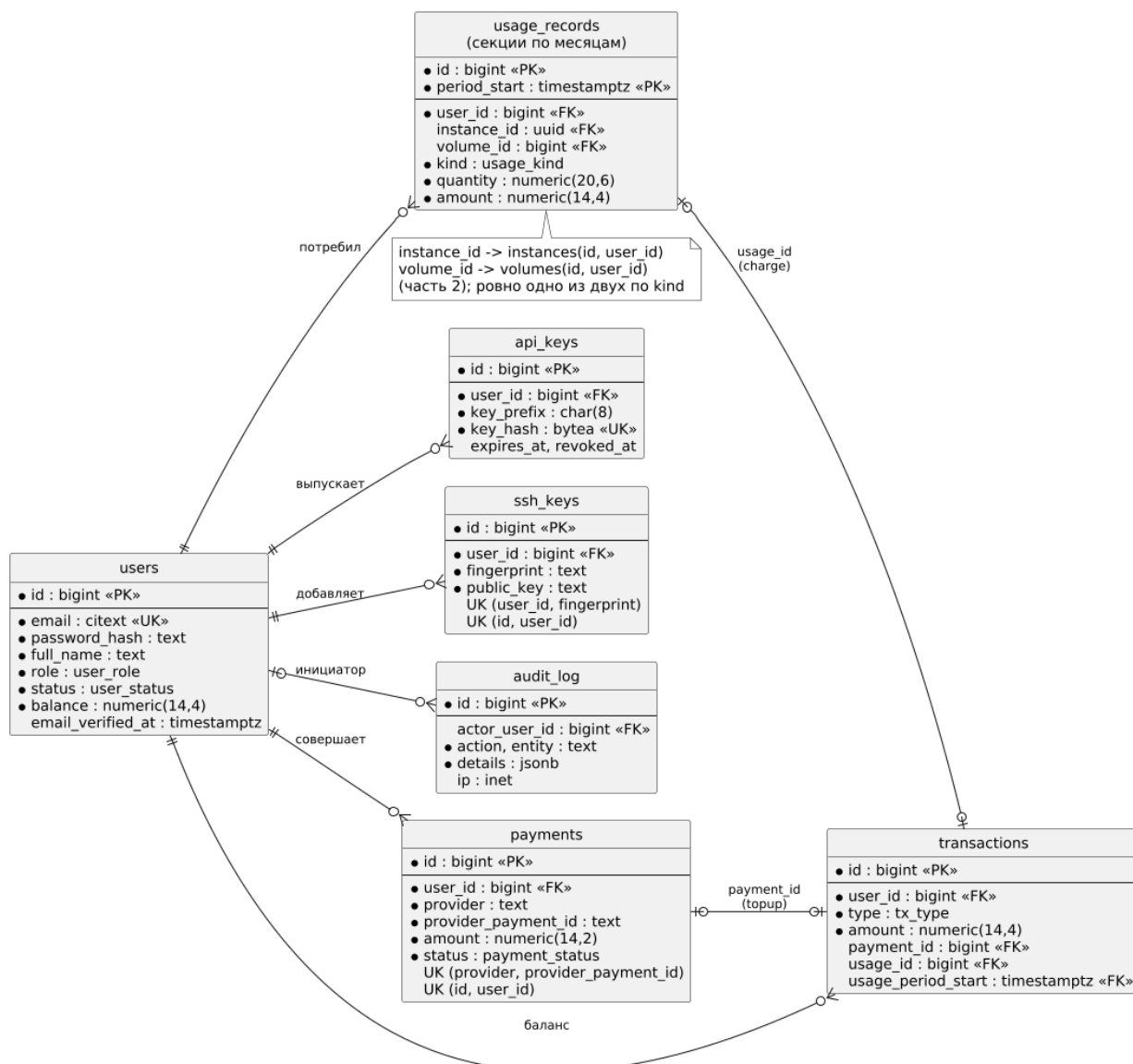


Рисунок 1 — Логическая ER-модель: пользователи, платежи и биллинг

Таблица и назначение	Основные поля и типы	Ключи и существенные ограничения
gpu_models: модели	name text, vram_gb smallint, fp32_tflops numeric(8,2)	UQ name; положительные характеристики
nodes: серверы	datacenter_id bigint, hostname text, status node_status	FK datacenters; UQ hostname
gpus: устройства	node_id bigint, gpu_model_id bigint, slot_index smallint, serial text	FK nodes, gpu_models; UQ serial и слота ноды
gpu_prices: цены GPU	gpu_model_id bigint, price_per_hour numeric(14,4), valid_during tstzrange	FK площадки и модели; цена > 0; EXCLUDE периодов
storage_prices: цены хранения	datacenter_id bigint, price_per_gb_month numeric(14,4), valid_during tstzrange	FK datacenters; цена > 0; EXCLUDE периодов
templates: шаблоны	owner_id bigint, docker_image text, default_ports text	FK users; публичный ⇔ owner_id отсутствует
volumes: тома	user_id bigint, size_gb integer, last_billed_at timestamptz	FK владельца и площадки; размер 10–10000 ГБ
instances: аренда	id uuid, user_id bigint, status instance_status, price_per_hour_snapshot numeric(14,4)	PK uuid; FK объектов; составные FK тома и SSH-ключа с user_id; 1–8 GPU
instance_gpus: выделение	instance_id uuid, gpu_id bigint, allocated_during tstzrange	FK instances, gpus; EXCLUDE пересечений одной GPU
instance_env_vars: секреты	instance_id uuid, name text, value_encrypted bytea	PK (instance_id, name); FK instances
usage_records: потребление	id bigint, period_start timestamptz, kind usage_kind, amount numeric(14,4)	PK (id, period_start); составные FK объекта и владельца; выбран ровно один ресурс
transactions: проводки	user_id bigint, type tx_type, amount numeric(14,4)	FK users, payments, usage_records; знак суммы по типу; одно зачисление/списание на источник
payments: платежи	user_id bigint, provider_payment_id text, amount numeric(14,2)	FK users; UQ (provider, provider_payment_id)
audit_log: аудит	actor_user_id bigint, action text, details jsonb, ip inet	Необязательный FK users; журнал только дополняется

1.3.2 Связи и целостность

Пользователь имеет много ключей, платежей, томов и инстансов; дата-центр — много нод, нода — много GPU. Инстанс относится к одной ноде и шаблону; необязательный том подключается максимум к одному живому инстансу, что обеспечивает частичный уникальный индекс. Связь инстансов и GPU — M:N через `instance_gpus`: одна GPU обслуживает разные инстансы в разные периоды.

Составные FK защищают владельца тома, SSH-ключа, платежа и объекта потребления. Однако FK `transactions → usage_records` содержит только `(usage_id, usage_period_start)`, без `user_id`: владельца списания согласует функция биллинга. Прямой INSERT в журнал приложению не разрешён.

EXCLUDE USING gist запрещает пересечение периодов выделения одной GPU и периодов одной ценовой позиции. Триггеры проверяют модель и ноду выделенной GPU, соответствие площадок тома и инстанса, переходы статусов и неизменяемые атрибуты. Проводки, потребление и аудит защищены от изменения и удаления; баланс меняется триггером вставки проводки.

1.3.3 Выбор типов

Для денег выбран точный `numeric`, поскольку округления `float` неприемлемы. Для времени — `timestamp_tz`, для периодов — `tstzrange` с границами `[]`, позволяющими соседним интервалам соприкасаться. `citext` обеспечивает уникальность email без учёта регистра, `uuid` используется для идентификатора инстанса в URL. `bytea` хранит хэши и шифртекст, `jsonb` — изменяемый состав событий аудита, `inet` — проверяемый IP-адрес. Статусы заданы перечислениями; обязательность определяется смыслом поля [4].

1.4 Оптимизация производительности

Индексы выбираются под запросы, поскольку каждый дополнительный индекс увеличивает стоимость записи. В миграции `0007_indexes.sql` предусмотрены:

- B-tree (`user_id`, `created_at` DESC, `id` DESC) для истории проводок и пагинации по последней записи;
- частичные индексы активных инстансов и открытых выделений GPU;
- (`user_id`, `period_start`) для потребления клиента и BRIN по времени для крупных упорядоченных секций;
- GIN по `audit_log.details` для поиска содержимого событий; GiST обслуживает ограничения периодов.

`usage_records` разделена по месяцам: фильтр времени исключает ненужные секции [4]. Баланс хранится как агрегат проводок, `mv_daily_revenue` — как материализованная витрина выручки. Функция обновления витрины реализована, запуск по расписанию ещё требует приложения или планировщика.

Redis 7 планируется для каталога GPU и цен с ограниченным сроком жизни и сбросом после изменения. Баланс и авторитетный статус инстанса кэшировать рискованно из-за устаревания. Эффект SQL-мер измерен в П4 через `EXPLAIN (ANALYZE, BUFFERS)`; эффект Redis пока только оценён, работающего кэша нет.

Запуск, пополнение и биллинг выполняются транзакционно; блокировки и `SKIP LOCKED` защищают конкурентное выделение ресурсов. Уникальные ключи источников препятствуют повторному пополнению и списанию.

1.5 Нормализация и денормализация

Рассмотрим строку счёта (`instance_id`, `period_start`, `email`, `full_name`, `gpu_model`, `vram_gb`, `period_end`, `amount`) с ключом (`instance_id`, `period_start`). В 1НФ одна строка соответствует одному интервалу вместо вложенного списка интервалов. Для 2НФ сведения об инстансе выносятся отдельно: они зависят только от `instance_id`, то есть части ключа. Для 3НФ устраняются зависимости `email` → `full_name` и `gpu_model` → `vram_gb`: появляются `users` и `gpu_models`. Это предотвращает противоречивые копии профиля и характеристик [2, 3].

ДЦ инстанса определяется через ноду и отдельно не хранится; несколько GPU представлены строками связи. Исключение — `templates.default_ports`: список портов в строке нарушает 1НФ, но передаётся целиком и не участвует в фильтрации.

Денормализация	Польза и последствия
<code>users.balance</code>	Быстрая проверка средств; согласуется триггером с неизменяемым журналом
<code>usage_records.user_id</code>	Фильтрация и RLS без соединений; согласуется составными FK
<code>quantity, amount, сумма списания</code>	Готовые расчёты и отчёты; потребление и проводку создаёт одна транзакция
<code>instances.gpu_model_id</code>	Аналитика без размножения строк выделениями; соответствие GPU проверяет триггер
<code>last_billed_at</code>	Быстрая граница учёта; обновляется под блокировкой, повтор интервала запрещён
<code>mv_daily_revenue</code>	Быстрый дашборд; данные устаревают до следующего обновления

Снимок цены — самостоятельный факт условий запуска. При возобновлении он обновляется, а суммы прошлых интервалов сохраняются в потреблении. Денормализация увеличивает сложность записи, поэтому нужны сверка баланса с журналом и проверка витрины против исходных данных.

1.6 Безопасность данных

`gpu_rent_owner` владеет схемой и выполняет миграции; `gpu_rent_app` получает ограниченные права таблиц, столбцов и функций; `gpu_rent_analyst` читает маскированное представление и агрегаты. Десять политик RLS на девяти таблицах используют транзакционный контекст `app.user_id`; функции `SECURITY DEFINER` имеют фиксированный `search_path` [5]. RLS защищает от пропущенного фильтра, но скомпрометированная роль приложения может подменить контекст. Таблицы `users` и `templates` RLS не закрыты.

Схема хранит `bcrypt`-хэш пароля и 32-байтовый `SHA-256`-хэш API-ключа, проверяя формат и длину; полный прикладной процесс создания и проверки хэшей ещё не реализован. Переменные окружения шифруются `pgp_sym_encrypt`, ключ поступает извне и в БД не сохраняется [6]. Аудит

фиксирует изменения пользователей, цен GPU и статусов инстансов; аудит цен хранения пока отсутствует.

TLS, SCRAM, шифрование диска и резервных копий, мониторинг относятся к плану эксплуатации. На стенде TLS отключён; защищённые носители и копии не настроены. Также не реализованы фиксация согласия и обезличивание аккаунта. Неизменяемость денежных журналов требует заранее согласовать архивирование и сроки хранения: FK проводок препятствует удалению связанных секций потребления.

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Какие основы проектирования баз данных вы усвоили? Сущности, атрибуты, кардинальность связей, ключи и ограничения целостности. Модель проходит от смысла предметной области к таблицам и физической реализации.

Могли бы вы описать процесс анализа требований и определения сущностей для базы данных? Для каждого сценария определяются необходимые факты, их источники и правила изменения. Так регистрация даёт пользователя, аренда — инстанс и выделение GPU, оплата — платёж и проводку.

Какие основные принципы нормализации баз данных вы понимаете? 1НФ исключает повторяющиеся группы, 2НФ — частичные зависимости, 3НФ — транзитивные. Это уменьшает дублирование и аномалии изменения данных.

Какие ключевые аспекты проектирования баз данных для веб-приложений вы изучили? Связь модели с требованиями, точность денег, конкурентность и рост данных. Изоляция пользователей и проверяемые ограничения должны учитываться вместе с запросами приложения.

Могли бы вы рассказать о методах оптимизации производительности баз данных? Индексы ускоряют конкретные выборки, секционирование сокращает просмотр данных, агрегаты и кэш уменьшают повторные вычисления. Пользу оценивают планами выполнения и измерениями, учитывая стоимость записи и устаревание.

Какие аспекты безопасности данных были учтены при проектировании базы данных? Права ролей и столбцов, RLS, хранение хэшей, шифрование секретов и аудит. Защита каналов, дисков и копий предусмотрена планом эксплуатации.

Можете ли вы объяснить основные концепции проектирования баз данных для монолитных веб-приложений? Единая БД является контрактом модулей монолита. Связанные действия можно выполнить одной ACID-транзакцией, сохраняя денежные и ресурсные инварианты.

Какие методы оптимизации производительности баз данных вы рассмотрели и применили? Применены частичные и составные индексы, BRIN, GIN, GiST, месячные секции и агрегаты. SQL-оптимизации проверены в П4; Redis остаётся планом.

Какие решения по нормализации и денормализации данных были приняты в вашем проекте? Пользователи, модели, площадки и выделения разделены; список портов оставлен исключением из 1НФ. Баланс, владелец потребления, расчётные суммы и витрина хранятся ради чтения с механизмами согласования.

Можете ли вы предоставить подробное описание процесса проектирования базы данных для монолитного веб-приложения? Требования преобразованы в 18 сущностей, затем построены ER-модели и определены атрибуты, ключи и ограничения. После проверки нормализации выбраны оптимизации и меры безопасности; реализация описана в П4.

Какие методы и инструменты вы использовали для оптимизации производительности вашей базы данных? PostgreSQL 16, индексы, секции, материализованное представление и EXPLAIN (ANALYZE, BUFFERS). Сценарии воспроизведения находятся в db/bench и db/scripts, Redis ещё не используется.

Какие меры безопасности данных были реализованы в вашей базе данных, и как они обеспечивают защиту информации? Миграции ограничивают права, фильтруют строки через RLS, защищают журналы и шифруют секреты окружения. Прикладная аутентификация и эксплуатационная защита остаются следующими этапами.

ЗАКЛЮЧЕНИЕ

Для GPU Rent построены концептуальная и логическая модели с 18 предметными сущностями. Определены ключи, типы, ограничения, методы оптимизации и защиты; денормализации снабжены механизмами согласования. Полный SQL сохранён в миграциях, физические проверки и измерения — в П4. Открыты эксплуатационная защита, прикладная аутентификация, агент ноды, согласие и обезличивание аккаунта, политика долга и накопительное округление.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Chen P. P.-S. The Entity-Relationship Model — Toward a Unified View of Data // ACM Transactions on Database Systems. — 1976. — Vol. 1, No. 1. — P. 9–36.
2. Дейт К. Дж. Введение в системы баз данных. — 8-е изд. — М.: Вильямс, 2005.
3. Kent W. A Simple Guide to Five Normal Forms in Relational Database Theory // Communications of the ACM. — 1983. — Vol. 26, No. 2. — P. 120–125.
4. PostgreSQL 16 Documentation. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 30.09.2026).
5. PostgreSQL 16 Documentation: Row Security Policies. — URL: <https://www.postgresql.org/docs/16/ddl-rowsecurity.html> (дата обращения: 30.09.2026).
6. PostgreSQL 16 Documentation: pgcrypto. — URL: <https://www.postgresql.org/docs/16/pgcrypto.html> (дата обращения: 30.09.2026).